

Kubernetes ConfigMap Update Scenarios

Scenario 1: ConfigMap as Environment Variables

In this example, the ConfigMap values are injected into the container as environment variables.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_NAME: "my-app"
  APP_MODE: "production"
  DB_HOST: "mysql.example.com"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: nginx:1.27
          envFrom:
            - configMapRef:
                name: app-config
```

Apply it:

```
kubectl apply -f app.yaml
```

Check the ConfigMap:

```
kubectl get configmap app-config -o yaml
```

Update the ConfigMap:

```
kubectl edit configmap app-config
```

For example:

```
data:
  APP_NAME: "my-new-app"
  APP_MODE: "development"
  DB_HOST: "new-db.example.com"
```

Important: Existing Pods keep the old environment-variable values.

Restart the Deployment:

```
kubectl rollout restart deployment my-app
```

New Pods will then receive the updated ConfigMap values.

Scenario 2: ConfigMap Mounted as a Volume

Here the ConfigMap is mounted as files inside the container.

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```

name: app-config-file
data:
  application.properties: |
    app.name=my-app
    app.mode=production
    db.host=mysql.example.com
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-file
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app-file
  template:
    metadata:
      labels:
        app: my-app-file
    spec:
      containers:
        - name: my-app
          image: nginx:1.27
          volumeMounts:
            - name: config-volume
              mountPath: /etc/myapp
      volumes:
        - name: config-volume
          configMap:
            name: app-config-file

```

The file will appear inside the container as:

```
/etc/myapp/application.properties
```

Update the ConfigMap:

```
kubectl edit configmap app-config-file
```

Kubernetes will eventually update the mounted file in the existing Pods.

Check it with:

```
kubectl exec <pod-name> -- cat /etc/myapp/application.properties
```

Note: The application itself must be capable of detecting/reloading the changed file. Kubernetes does not automatically restart the application.

The Key Difference

ConfigMap	Environment Variable	Volume Mount
Existing Pod	Keeps old value	File gets updated
After ConfigMap update	Restart required	App must reload

Production Pattern

```

kubectl edit configmap app-config
kubectl rollout restart deployment my-app
kubectl rollout status deployment my-app

```

This gives you a controlled rolling restart rather than manually deleting Pods.

Important Interview Point

ConfigMap mounted as a volume can be updated in an existing Pod, while ConfigMap values injected as environment variables require the Pod to be recreated/restarted to get the new values.